

Evidencia 1: Ordenamiento y búsqueda en logs de seguridad

Documento de evidencias de pruebas

Diego Contreras Alvarez — A00845716

Programación de Estructuras de Datos y Algoritmos Fundamentales (TC1031)

Grupo 607

Profesor: David Alonso Cantú Delgado

Fecha: 20 de septiembre de 2026

Plan de pruebas

El objetivo de las pruebas es comprobar que la aplicación lee correctamente los dos archivos de log, que todos los algoritmos de ordenamiento producen la misma secuencia cronológica y que la búsqueda por rango regresa exactamente los registros que debe, sin perder ni repetir ninguno.

Primero se valida la lectura de datos, confirmando que se cargan los 6,818 registros de cada archivo y que el programa responde con un mensaje claro cuando un archivo no existe o una línea viene mal formada. Esto importa porque cualquier error en la lectura contamina todo lo que sigue.

Después se valida el ordenamiento revisando que el primer y el último registro de output607.txt sean los correctos, que el archivo conserve el formato original y que todos los algoritmos generen la misma salida. También se mide el tiempo de cada algoritmo en los dos archivos para comparar su comportamiento con datos desordenados y casi ordenados.

La búsqueda por rango se prueba con casos que tienen un resultado conocido, como un rango vacío, rangos que empiezan o terminan exactamente en el primer o el último registro, un rango que cubre todos los datos y rangos cuyos límites caen en fechas repetidas. Estos últimos son los más importantes porque los dos archivos traen 18 pares de registros con la misma fecha y hora, y una búsqueda binaria mal planteada puede perder uno de los dos. La instrucción pide documentar este caso con log607-2.txt.

Por último se prueban entradas inválidas en el menú y en las fechas, para confirmar que el programa explica qué salió mal y deja volver a intentar sin cerrarse.

Casos de prueba

Fase 1: lectura de los archivos y validación de entradas.

ID	Qué se prueba	Entrada	Esperado	Obtenido	¿Pasó?
P01	Lectura completa del archivo desordenado	data/log607-1.txt	6818 registros	6818 registros. Clave del primero 20240929143738	Sí
P02	Lectura completa del archivo casi ordenado	data/log607-2.txt	6818 registros	6818 registros. Clave del primero 20240908002243	Sí
P03	Archivo inexistente	data/no-existe.txt	Mensaje con la ruta, sin cerrarse	Avisa con la ruta que intentó abrir y sigue con el resto	Sí
P04	Líneas vacías y mal formadas	data/prueba-malformada.txt	2 registros y 4 avisos (líneas 2, 3, 4 y 6)	2 registros. Se ignoraron 4 líneas de 6: la 2 vacía, y la 3, 4 y 6 con formato inválido	Sí
P05	Fecha con día que no existe en ese mes	Línea 6 de data/prueba-malformada.txt (Feb 30 2025)	Línea 6 ignorada por formato inválido	Línea 6 ignorada por formato inválido. Feb 29 2024 sí se acepta porque es bisiesto	Sí
P06	Letras en el menú	abc	Mensaje de error y vuelve a preguntar	Entrada invalida. Escribe un numero entre 0 y 3. Vuelve a pedir la opcion	Sí

ID	Qué se prueba	Entrada	Esperado	Obtenido	¿Pasó?
P07	Número fuera de rango	7	Mensaje de error indicando el rango válido	Entrada invalida. Escribe un numero entre 0 y 3.	Sí
P08	Entrada vacía y formatos mixtos	Enter, 1.5, 2abc	Mensaje de error en cada una	Las tres dan Entrada invalida y el menu vuelve a preguntar	Sí
P09	Número demasiado grande	9999999999	Mensaje de error sin que el programa se cierre	Ese numero es demasiado grande. El programa sigue abierto, no truena stoi	Sí
P10	Buscar antes de ordenar	Opción 2 sin haber ordenado	Mensaje de que primero hay que ordenar	Primero ordena un archivo con la opcion 1. La busqueda binaria solo funciona sobre datos ordenados	Sí
P11	Entrada cerrada con Ctrl + D	Ctrl + D en el menú	Mensaje de salida sin ciclo infinito	Entrada terminada, saliendo del programa. Termina con código 0	Sí
P12	Espacios sobrantes alrededor del número	(espacios)1(espacios)	Se acepta como 1	Carga log607-1.txt igual que si se escribiera 1 sin espacios	Sí
P13	Ordenamiento del archivo desordenado	Bubble sort en log607-1	Verificacion correcta y 6818 registros	Ordenado correctamente. 100.201 ms	Sí
P14	Ordenamiento del archivo casi ordenado	Bubble sort en log607-2	Verificacion correcta y tiempo mucho menor que en P13	Ordenado correctamente. 0.268 ms, unas 370 veces mas rapido	Sí
P15	Formato y contenido de output607.txt	head, tail y wc -l sobre la salida	Primer registro Sep 08 2024 00:22:43, ultimo Sep 07 2026 23:50:40, 6818 lineas, mismo formato que la entrada	Los tres coinciden. Los 6818 registros son los mismos de la entrada, solo reordenados	Sí
P16	Cada corrida ordena datos frescos	Dos corridas seguidas de bubble sort en log607-1	Tiempos parecidos en ambas	111.330 ms y 122.780 ms. Ninguna baja a cero, la copia si se rehace	Sí
P17	Estabilidad en fechas repetidas	Los 18 pares duplicados de log607-1	Cada par conserva el orden en que venia en el archivo	Los 18 pares conservan su orden. Bubble sort solo intercambia con menor estricto	Sí
P18	Todos los algoritmos ordenan correctamente	Los 8 algoritmos en log607-1 y log607-2	Verificacion correcta en las 16 corridas	Las 16 corridas verifican. Las 8 salidas tienen los	Sí

ID	Qué se prueba	Entrada	Esperado	Obtenido	¿Pasó?
				mismos 6818 registros de la entrada	
P19	Estabilidad de los algoritmos	diff de cada salida contra la de bubble sort en log607-1	0 diferencias en insertion y merge. En los demas solo difieren lineas con fechas repetidas	Insertion y merge dan 0. Selection 8, quick mediana 4, swap 11, shell 6 y quick pivote final 10, todas con fecha repetida	Sí
P20	Quick sort con pivote al final no desborda la pila	Variante en log607-2 (casi ordenado)	Termina sin cerrarse pese a la recursion profunda	Termina y verifica. Tarda 38.057 ms contra 1.193 ms en el desordenado	Sí
P21	Predicción obligatoria antes de cada corrida	Cualquier corrida	Pide categoría y razón antes de ordenar	Las dos preguntas salen siempre, después de elegir archivo y algoritmo y antes de medir	Sí
P22	Predicción y razón inválidas	Categoría 7, 4, abc y razón vacía o de puros espacios	Mensaje de error y vuelve a preguntar en cada caso	Entrada invalida. Escribe un numero entre 1 y 3. La razón vacía da El texto no puede ir vacio	Sí
P23	Predicción que coincide	Lento con bubble sort en log607-1	Coincidió: si	102.554 ms, clasificado lento. Coincidio: si	Sí
P24	Predicción que no coincide	Medio con insertion sort en log607-2	Coincidió: no, resultado rápido	1.204 ms, clasificado rapido. Coincidio: no	Sí
P25	Razón con comas y comillas	es rapido, porque "casi" esta ordenado	El CSV la guarda completa y el historial se ve alineado	En el CSV queda "es rapido, porque ""casi"" esta ordenado". El historial no se desacomoda	Sí
P26	Historial vacío y con corridas	Opción 3 antes y después de correr	Mensaje de sin corridas, luego tabla numerada	Aun no hay corridas registradas con el CSV borrado. Después, tabla con las 4 corridas	Sí
P27	El historial persiste entre ejecuciones	Cerrar, abrir y opción 3	Aparecen las corridas anteriores	Las 4 corridas vienen de 4 ejecuciones distintas y el CSV tiene un solo encabezado	Sí
P28	Duplicados en ambos límites	Oct 02 2024 23:04:24 a la misma fecha	2 registros, ambos con esa fecha	2 registros. Salen los dos: 10.14.121.223 y 10.14.47.219	Sí

ID	Qué se prueba	Entrada	Esperado	Obtenido	¿Pasó?
P29	Duplicados en inicio y fin	Feb 14 2026 14:43:30 a Feb 14 2026 22:00:18	5 registros	5 registros. Entran completos los pares de los dos extremos	Sí
P30	Rango con repetidos adentro	Dec 16 2025 00:00:00 a Dec 16 2025 23:59:59	13 registros	13 registros, con los dos pares repetidos de ese día	Sí
P31	Rango vacío dentro del periodo	Jul 11 2025 00:00:00 a Jul 11 2025 09:00:00	0 registros, range607.txt vacío	0 registros. El archivo se genera con 0 líneas	Sí
P32	Rango vacío fuera del periodo	Jan 01 2023 00:00:00 a Dec 31 2023 23:59:59	0 registros	0 registros, sin error. Avisa que revise el periodo del archivo	Sí
P33	Bordes exactos	Primer y último registro por separado	1 registro cada uno	Sep 08 2024 00:22:43 da 1. Sep 07 2026 23:50:40 a Dec 31 2026 da 1	Sí
P34	Rango que cubre todo	Jan 01 2020 00:00:00 a Dec 31 2030 23:59:59	6818 registros	6818 registros. No sale 6817, el límite superior está bien	Sí
P35	Fechas con formato inválido	Foo 32 2025 25:61:00, Feb 30 2025 10:00:00, vacío y espacios	Mensaje de error y vuelve a pedir en cada caso	Las cuatro se rechazan. Fecha invalida en dos, El texto no puede ir vacío en dos	Sí
P36	Inicio posterior al fin	Inicio Dec 16 2025, fin Jan 15 2025	Mensaje de error y vuelve a pedir	La fecha de inicio no puede ser posterior a la de fin. Pide las dos de nuevo	Sí
P37	Coherencia entre pantalla y archivo	wc -l out/range607.txt en los 8 rangos	El archivo tiene tantas líneas como registros anunció la pantalla	Coinciden en los 8 casos: 2, 5, 13, 0, 0, 1, 1 y 6818	Sí
P38	Salidas identificadas por archivo	Ordenar y buscar en los dos archivos	Existen las cuatro copias con contenido correcto	Las cuatro existen. output607_log1 y _log2 tienen los mismos 6818 registros y difieren en 20 líneas, todas de fechas repetidas	Sí
P39	Compilación sin advertencias	g++ -std=c++20 -O2 -Wall -Wextra	Cero advertencias	Cero. También queda limpio agregando -Wpedantic y -Wshadow	Sí
P40	Ejecución desde carpeta equivocada	Correr el binario desde la raíz del repositorio	Mensaje claro de que no encontró los datos y que hay que ejecutar desde Evidence1	Error: no se pudo abrir el archivo data/log607-1.txt. Revisa que exista y que lo estes corriendo desde la carpeta Evidence1	Sí

ID	Qué se prueba	Entrada	Esperado	Obtenido	¿Pasó?
P41	El script run.sh funciona desde cualquier carpeta	./scripts/run.sh desde Evidence1 y desde la raíz	Compila y ejecuta igual en los dos casos	Igual en ambos. El script se mueve solo a Evidence1 antes de compilar	Sí

Captura P01: Lectura completa del archivo desordenado

```
Option: 1
Que archivo quieres usar?
1. log687-1.txt (desordenado)
2. log687-2.txt (casi ordenado)
0. Regresar al menu
Archivo: 1
Se cargaron 6818 registros de log687-1.txt (desordenado).
```

Captura P02: Lectura completa del archivo casi ordenado

```
Option: 1
Que archivo quieres usar?
1. log687-1.txt (desordenado)
2. log687-2.txt (casi ordenado)
0. Regresar al menu
Archivo: 2
Se cargaron 6818 registros de log687-2.txt (casi ordenado).
```

Captura P03: Archivo inexistente

```
diegocontreras@Diegos-MacBook-Pro-181: Evidence1 % clear
cd ~/Developer/C.HTY.TC1831.687.2613
./Evidences/Evidence1/build/app
=====
1. Ordenar un archivo
2. Buscar por rango de fechas
3. Ver historial de corridas
0. Salir
Option: 1
Que archivo quieres usar?
1. log687-1.txt (desordenado)
2. log687-2.txt (casi ordenado)
0. Regresar al menu
Archivo: 1
Error: no se pudo abrir el archivo "data/log687-1.txt".
Revisa que exista y que lo estes corriendo desde la carpeta Evidence1.
=====
Analizador de logs 687
=====
1. Ordenar un archivo
2. Buscar por rango de fechas
3. Ver historial de corridas
0. Salir
Option: 0
Hasta luego.
*diegocontreras@Diegos-MacBook-Pro-181 C.HTY.TC1831.687.2613 % ./Evidences/Evidence1/scripts/run.sh
=====
Analizador de logs 687
=====
1. Ordenar un archivo
2. Buscar por rango de fechas
3. Ver historial de corridas
0. Salir
Option: 0
Hasta luego.
^Cdiegocontreras@Diegos-MacBook-Pro-181 C.HTY.TC1831.687.2613 % █
```

Captura P04: Líneas vacías y mal formadas

```
Option: 1
Que archivo quieres usar?
1. log687-1.txt (desordenado)
2. log687-2.txt (casi ordenado)
3. prueba-malformada.txt (archivo de prueba de manejo de errores)
0. Regresar al menu
Archivo: 3
Línea 2 ignorada porque esta vacia.
Línea 3 ignorada por formato invalido.
Línea 4 ignorada por formato invalido.
Línea 6 ignorada por formato invalido.
Se ignoraron 4 líneas de 6 en total.
Se cargaron 2 registros de prueba-malformada.txt (archivo de prueba de manejo de errores).
```

Captura P05: Fecha con día que no existe en ese mes

```
Option: 1
Que archivo quieres usar?
1. log687-1.txt (desordenado)
2. log687-2.txt (casi ordenado)
3. prueba-malformada.txt (archivo de prueba de manejo de errores)
0. Regresar al menu
Archivo: 3
Línea 2 ignorada porque esta vacia.
Línea 3 ignorada por formato invalido.
Línea 4 ignorada por formato invalido.
Línea 6 ignorada por formato invalido.
Se ignoraron 4 líneas de 6 en total.
Se cargaron 2 registros de prueba-malformada.txt (archivo de prueba de manejo de errores).
```

Captura P06 a P09: entradas inválidas en el menú (una sola sesión)

```
diegocontreras@Diegos-MacBook-Pro-181 Evidence1 % ./build/app
```

```
=====
Analizador de logs 687
=====
1. Ordenar un archivo
2. Buscar por rango de fechas
3. Ver historial de corridas
0. Salir
Opcion: abc
Entrada invalida. Escribe un numero entre 0 y 3.
Opcion: 7
Entrada invalida. Escribe un numero entre 0 y 3.
Opcion:
Entrada invalida. Escribe un numero entre 0 y 3.
Opcion: 1.5
Entrada invalida. Escribe un numero entre 0 y 3.
Opcion: 2abc
Entrada invalida. Escribe un numero entre 0 y 3.
Opcion: 9999999999
Ese numero es demasiado grande. Escribe un numero entre 0 y 3.
Opcion: █
```

Captura P10: buscar antes de ordenar

```
Opcion: 2
Primero ordena un archivo con la opcion 1. La busqueda binaria solo funciona sobre datos ordenados.

=====
Analizador de logs 687
=====
1. Ordenar un archivo
2. Buscar por rango de fechas
3. Ver historial de corridas
0. Salir
Opcion: █
```

Captura P11 y P12: Ctrl + D y número con espacios

```
Opcion: 1
Que archivo quieres usar?
1. log687-1.txt (desordenado)
2. log687-2.txt (cast ordenado)
0. Regresar al menu
Archivo: 1
Se cargaron 6818 registros de log687-1.txt (desordenado).

Que algoritmo quieres usar?
1. Bubble sort
2. Selection sort
3. Insertion sort
4. Merge sort
5. Quick sort (mediana de tres)
6. Swap sort
7. Shell sort
8. Quick sort (pivotar al final, variante)
0. Regresar al menu
Algoritmo: 0

=====
Analizador de logs 687
=====
1. Ordenar un archivo
2. Buscar por rango de fechas
3. Ver historial de corridas
0. Salir
Opcion: "D
Entrada terminada, saliendo del programa.
diegocontreras@Diegos-MacBook-Pro-181 Evidence1 % █
```

Captura P13 y P14: bubble sort en los dos archivos

```

=====
Algoritmo:      Bubble sort
Archivo:        log607-1.txt (desordenado)
Registros:     6819
Tiempo:         109.583 ns
Complejidad:    mejor caso O(n), peor caso O(n^2)
Estable:        si
Prediccion:     lento
Resultado:       lento
Coincidio:      si
Verificacion:    datos ordenados correctamente
Salida guardada en out/output607.txt
Copia guardada en out/output607_log1.txt
Corrida guardada en out/corridas.csv
=====

=====
Analizador de logs 607
=====
1. Ordenar un archivo
2. Buscar por rango de fechas
3. Ver historial de corridas
0. Salir
Option: 1

Que archivo quieres usar?
1. log607-1.txt (desordenado)
2. log607-2.txt (casi ordenado)
0. Regresar al menu
Archivo: 2
Se cargaron 6819 registros de log607-2.txt (casi ordenado).

Que algoritmo quieres usar?
1. Bubble sort
2. Selection sort
3. Insertion sort
4. Merge sort
5. Quick sort (mediana de tres)
6. Swap sort
7. Shell sort
8. Quick sort (pivote al final, variante)
0. Regresar al menu
Algoritmo: 1

Vas a ordenar 6819 registros de log607-2.txt (casi ordenado) con Bubble sort.

Antes de ordenar, haz tu prediccion.
Que tan rapido crees que sera este algoritmo con este archivo?
1. Rapido (menos de 10 ns)
2. Medio (entre 10 y 60 ns)
3. Lento (mas de 60 ns)
Prediccion: 1
Por que? Menciona el tamaño de los datos y que tan ordenado esta el archivo: el archivo ya viene casi ordenado y la bandera corta las pasadas muy pronto

=====
Algoritmo:      Bubble sort
Archivo:        log607-2.txt (casi ordenado)
Registros:     6819
Tiempo:         0.545 ns
Complejidad:    mejor caso O(n), peor caso O(n^2)
Estable:        si
Prediccion:     rapido
Resultado:       rapido
Coincidio:      si
Verificacion:    datos ordenados correctamente
Salida guardada en out/output607.txt
Copia guardada en out/output607_log2.txt
Corrida guardada en out/corridas.csv
=====

```

Captura P15: head, tail y wc -l de out/output607.txt

```

• diegocontreras@Diegos-MacBook-Pro-181 Evidence1 % head -3 out/output607.txt
tail -3 out/output607.txt
wc -l out/output607.txt
wc -l out/output607.txt
Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication
Sep 08 2024 04:37:37 10.14.248.79 Configuration errors
Sep 08 2024 04:44:15 10.14.135.12 Application error
Sep 07 2026 17:30:32 10.14.109.102 Port Scanning Activity
Sep 07 2026 23:08:26 10.14.227.119 Data Exfiltration Attempts
Sep 07 2026 23:50:40 10.14.178.169 Security breaches
6819 out/output607.txt
13 out/output607.txt
• diegocontreras@Diegos-MacBook-Pro-181 Evidence1 %

```

Captura P16: dos corridas seguidas en log607-1

```

=====
Algoritmo:      Bubble sort
Archivo:        log607-1.txt (desordenado)
Registros:      6819
Tiempo:         113.570 ns
Complejidad:    mejor caso O(n), peor caso O(n^2)
Estable:        si
Prediccion:     lento
Resultado:       lento
Coincidio:      si
Verificacion:   datos ordenados correctamente
Salida guardada en out/output607.txt
Copia guardada en out/output607_log1.txt
Corrida guardada en out/corridas.csv
=====

=====
Analizador de logs 607
=====
1. Ordenar un archivo
2. Buscar por rango de fechas
3. Ver historial de corridas
0. Salir
Opcion: 1

Que archivo quieres usar?
1. log607-1.txt (desordenado)
2. log607-2.txt (casi ordenado)
0. Regresar al menu
Archivo: 1
Se cargaron 6819 registros de log607-1.txt (desordenado).

Que algoritmo quieres usar?
1. Bubble sort
2. Selection sort
3. Insertion sort
4. Merge sort
5. Quick sort (mediana de tres)
6. Swap sort
7. Shell sort
8. Quick sort (pivote al final, variante)
0. Regresar al menu
Algoritmo: 1

Vas a ordenar 6819 registros de log607-1.txt (desordenado) con Bubble sort.

Antes de ordenar, haz tu prediccion.
Que tan rapido crees que sera este algoritmo con este archivo?
1. Rapido (menos de 10 ns)
2. Medio (entre 10 y 60 ns)
3. Lento (mas de 60 ns)
Prediccion: 3
Por que? Menciona el tamaño de los datos y que tan ordenado esta el archivo: segunda corrida seguida, el tiempo debe parecerse al de la primera

=====
Algoritmo:      Bubble sort
Archivo:        log607-1.txt (desordenado)
Registros:      6819
Tiempo:         112.267 ns
Complejidad:    mejor caso O(n), peor caso O(n^2)
Estable:        si
Prediccion:     lento
Resultado:       lento
Coincidio:      si
Verificacion:   datos ordenados correctamente
Salida guardada en out/output607.txt
Copia guardada en out/output607_log1.txt
Corrida guardada en out/corridas.csv
=====

```

Captura P18: resumen de las corridas de los ocho algoritmos

#	Algoritmo	Archivo	Tiempo (ns)	Prediccion	Resultado	Coincidio
32	Bubble sort	log607-1.txt	104.510	lento	lento	si
33	Selection sort	log607-1.txt	22.008	lento	medio	no
34	Insertion sort	log607-1.txt	131.905	lento	lento	si
35	Merge sort	log607-1.txt	7.528	rapido	rapido	si
36	Quick sort (mediana de tres)	log607-1.txt	0.940	rapido	rapido	si
37	Swap sort	log607-1.txt	86.895	lento	lento	si
38	Shell sort	log607-1.txt	19.519	rapido	medio	no
39	Quick sort (pivote al final, variante)	log607-1.txt	2.012	rapido	rapido	si

Captura P19: diff de cada algoritmo contra bubble sort

```

❯ diegocontreras@Diegos-MacBook-Pro-101: Evidencia1 % clear
printf '\n\n\n\n\nprueba\n\n' | ./build/app > /dev/null
cp out/output607.txt /tmp/ref_bubble.txt
for alg in 2 3 4 5 6 7 8; do
  printf '\n\n\n\n\nprueba\n\n' | ./build/app > /dev/null
  echo "algoritmo $alg: $(diff /tmp/ref_bubble.txt out/output607.txt | grep -o ">") líneas distintas"
done
algoritmo 2: 8 líneas distintas
algoritmo 3: 0 líneas distintas
algoritmo 4: 0 líneas distintas
algoritmo 5: 4 líneas distintas
algoritmo 6: 11 líneas distintas
algoritmo 7: 6 líneas distintas
algoritmo 8: 10 líneas distintas
❯ diegocontreras@Diegos-MacBook-Pro-101: Evidencia1 %

```

Captura P20: quick sort con pivote al final en log607-2

Antes de ordenar, haz tu predicción.
 ¿Que tan rápido crees que será este algoritmo con este archivo?

1. Rápido (menos de 10 ms)
2. Medio (entre 10 y 60 ms)
3. Lento (mas de 60 ms)

Predicción: 3

Por que? Menciona el tamaño de los datos y que tan ordenado está el archivo: con el archivo casi ordenado el pivote es casi siempre el mayor y la partición queda vacía de un lado

```

Algoritmo: Dijkstra sort (pivotar al final, variante)
Archivo: log607-2.txt (casi ordenado)
Registros: 6818
Tiempo: 45.645 ms
Complejidad: mejor caso 0(n log n), peor caso 0(n^2)
Estable: no
Predicción: lento
Resultado: medio
Coincidió: no
Verificación: datos ordenados correctamente
Salida guardada en out/output607.txt
Copia guardada en out/output607_log2.txt
Corrida guardada en out/corridos.csv

```

```

Algoritmo: Bubble sort
Archivo: log687-1.txt (desordenado)
Registros: 6818
Tiempo: 109.503 ms
Complejidad: mejor caso O(n), peor caso O(n^2)
Estable: si
Predicción: lento
Resultado: lento
Coincidió: si
Verificación: datos ordenados correctamente
Salida guardada en out/output687.txt
Copia guardada en out/output687_log1.txt
Corrida guardada en out/corridas.csv

```

```
=====
Analizador de logs 687
=====
1. Ordenar un archivo
2. Buscar por rango de fechas
3. Ver historial de corridas
0. Salir
Option: 1
```

```

Due archivo quieres usar?
1. log607-1.txt (desordenado)
2. log607-2.txt (casi ordenado)
0. Regresar al menu
Archivo: 2
Se cargaron 6818 registros de log607-2.txt (casi ordenado).

```

```

Que algoritmo quieres usar?
1. Bubble sort
2. Selection sort
3. Insertion sort
4. Merge sort
5. Quick sort (mediana de tres)
6. Swap sort
7. Shell sort
8. Quick sort (pivote al final, variante)
9. Regresar al menu
Algoritmo: 1

```

Antes de ordenar, haz tu predicción.
 ¿Qué tan rápido crees que será este algoritmo con este archivo?

1. Rápido (menos de 18 ms)
2. Medio (entre 18 y 60 ms)
3. Lento (más de 60 ms)

Predicción: 1

Por qué? ¿Menciona el tamaño de los datos y que tan ordenado está el archivo: el archivo ya viene casi ordenado y la bandera corta las pasadas muy pronto

```

Algoritmo:      Bubble sort
Archivo:        log687-2.txt (cast. ordenado)
Registros:      6818
Tiempo:         0.545 ms
Complejidad:    mejor caso 0(n), peor caso 0(n^2)
Estable:        si
Predicción:     rápido
Resultado:      rápido
Coincidio:      si
Verificación:   datos ordenados correctamente
Salida guardada en out/output687.txt
Copia guardada en out/output687_log2.txt
Corrida guardada en out/corridas.csv

```

[illegible]

Captura P26 y P27: historial vacío y con las corridas acumuladas

```

Opcion: 3
# Algoritmo      Archivo      Tiempo (ms)  Prediccion  Resultado  Confidido
1 Bubble sort    log687-1.txt    195.697      lento        lento      si
2 Selection sort log687-1.txt    21.335        lento        medio      no
3 Insertion sort log687-1.txt    126.185      lento        lento      si
4 Merge sort     log687-1.txt    3.891         rapido      rapido     si
5 Quick sort (mediana de tres) log687-1.txt    0.953         rapido      rapido     si
6 Swap sort      log687-1.txt    87.424        lento        lento      si
7 Shell sort     log687-1.txt    7.832         rapido      rapido     si
8 Quick sort (pivot al final, variante) log687-1.txt    0.966         rapido      rapido     si
9 Bubble sort    log687-2.txt    0.298         rapido      rapido     si
10 Selection sort log687-2.txt    22.874        lento        medio      no
11 Insertion sort log687-2.txt    0.962         rapido      rapido     si
12 Merge sort     log687-2.txt    3.832         rapido      rapido     si
13 Quick sort (mediana de tres) log687-2.txt    0.675         rapido      rapido     si
14 Swap sort      log687-2.txt    15.488        lento        medio      no
15 Shell sort     log687-2.txt    5.779         rapido      rapido     si
16 Quick sort (pivot al final, variante) log687-2.txt    37.974        lento        medio      no
17 Merge sort     log687-2.txt    2.931         rapido      rapido     si
18 Merge sort     log687-1.txt    2.748         rapido      rapido     si
19 Bubble sort    log687-1.txt    189.583      lento        lento      si
20 Bubble sort    log687-2.txt    0.545         rapido      rapido     si
21 Bubble sort    log687-1.txt    113.570      lento        lento      si
22 Bubble sort    log687-1.txt    112.267      lento        lento      si

La razon de cada prediccion esta en out/corridos.csv.

```

Captura P28: duplicados en ambos límites, con los dos registros visibles

```

Opcion: 2

Hay 6618 registros ordenados.
Los datos van de Sep 08 2024 00:22:43 a Sep 07 2025 23:50:40.
Las fechas que escribas no tienen que existir en el archivo.
Fecha de Inicio: Oct 02 2024 23:04:24
Fecha de fin: Oct 02 2024 23:04:24

Registros encontrados: 2

Oct 02 2024 23:04:24 10.14.121.223 Changes in File Integrity
Oct 02 2024 23:04:24 10.14.47.219 Social engineering attacks
Resultado guardado en out/range67.txt
Copia guardada en out/range67_log1.txt

```

Captura P29: duplicados en el inicio y en el fin

```

Option: 2

Hay 6818 registros ordenados.
Los datos van de Sep 08 2004 00:22:43 a Sep 07 2006 22:39:40.
Las fechas que escribas no tienen que existir en el archivo.
Fecha de inicio: Feb 14 2006 14:43:30
Fecha de fin: Feb 14 2006 22:00:18

Registros encontrados: 5

Feb 14 2006 14:43:30 10.14.166.249 Malware Detection Alerts
Feb 14 2006 14:43:30 10.14.125.92 Changes in File Integrity
Feb 14 2006 21:42:51 10.14.194.38 SQL Injection
Feb 14 2006 22:00:18 10.14.147.50 SQL Injection
Feb 14 2006 22:00:18 10.14.8.91 Cross-Site Scripting

Resultado guardado en out/range687.txt
Copia guardada en out/range687_log1.txt

```

Captura P30: día con dos pares repetidos adentro

```

Option: 2

Hay 6818 registros ordenados.
Los datos van de Sep 08 2004 00:22:43 a Sep 07 2025 23:59:49.
Las fechas que escribas no tienen que existir en el archivo.
Fecha de Inicio: Dec 16 2025 00:00:00
Fecha de fin: Dec 16 2025 23:59:59

Registros encontrados: 13

Dec 16 2025 04:38:29 10.14.22.153 Application error
Dec 16 2025 05:01:57 10.14.215.155 SQL injection
Dec 16 2025 05:01:57 10.14.246.45 Anomalous User Activity
Dec 16 2025 07:24:21 10.14.73.05 Suspicious IP Addresses
Dec 16 2025 10:16:35 10.14.2.67 Application error
Dec 16 2025 10:16:35 10.14.130.127 Suspicious IP Addresses
Dec 16 2025 10:24:29 10.14.86.123 Application error
Dec 16 2025 11:50:49 10.14.40.194 Connection Failures
Dec 16 2025 13:19:24 10.14.112.284 Cross-Site Scripting
Dec 16 2025 15:54:49 10.14.230.195 Unauthorized Access Attempts
Dec 16 2025 16:29:42 10.14.216.64 Data Exfiltration Attempts
Dec 16 2025 20:35:56 10.14.72.88 Data corruption
Dec 16 2025 22:11:09 10.14.120.29 Weak or easily guessable passwords
Resultado guardado en out/range687.txt
Copia guardada en out/range687_lopi.txt

```

Captura P31 a P34: rangos vacíos, bordes y rango completo

```
Los datos van de Sep 08 2024 00:22:43 a Sep 07 2026 23:58:40.
Las fechas que escribas no tienen que existir en el archivo.
Fecha de inicio: Jan 01 2023 00:00:00
Fecha de fin: Dec 31 2023 23:59:59

Registros encontrados: 0
No hay registros en ese rango. Revisa que las fechas esten dentro del periodo que cubre el archivo.
Resultado guardado en out/range607.txt
Copia guardada en out/range607_log1.txt
```

```
=====
Analizador de logs 607
=====
1. Ordenar un archivo
2. Buscar por rango de fechas
3. Ver historial de corridas
0. Salir
Opcion: 2
```

```
Hay 6818 registros ordenados.
Los datos van de Sep 08 2024 00:22:43 a Sep 07 2026 23:58:40.
Las fechas que escribas no tienen que existir en el archivo.
Fecha de inicio: Sep 08 2024 00:22:43
Fecha de fin: Sep 08 2024 00:22:43
```

Registros encontrados: 1

```
Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication
Resultado guardado en out/range607.txt
Copia guardada en out/range607_log1.txt
```

```
Hay 6818 registros ordenados.
Los datos van de Sep 08 2024 00:22:43 a Sep 07 2026 23:58:40.
Las fechas que escribas no tienen que existir en el archivo.
Fecha de inicio: Jan 01 2020 00:00:00
Fecha de fin: Dec 31 2020 23:59:59
```

Registros encontrados: 6818

```
Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication
Sep 08 2024 04:37:37 10.14.248.79 Configuration errors
Sep 08 2024 04:44:15 10.14.135.12 Application error
Sep 08 2024 05:58:57 10.14.22.235 Changes in file integrity
Sep 08 2024 10:42:42 10.14.40.195 Data corruption
Sep 08 2024 10:47:53 10.14.24.223 Data corruption
Sep 08 2024 15:23:16 10.14.206.243 Weak or easily guessable passwords
Sep 08 2024 15:25:34 10.14.220.55 Broken authentication and session management
Sep 09 2024 01:51:20 10.14.27.106 Application error
Sep 09 2024 04:53:57 10.14.96.200 Suspicious IP addresses
... (6790 registros mas) ...
Sep 07 2026 04:21:30 10.14.40.44 Connection Failures
Sep 07 2026 04:39:50 10.14.43.154 Network connectivity issues
Sep 07 2026 08:19:52 10.14.237.112 Weak or easily guessable passwords
Sep 07 2026 11:42:30 10.14.229.232 Data Exfiltration Attempts
Sep 07 2026 11:52:19 10.14.40.210 Data corruption
Sep 07 2026 14:39:06 10.14.212.167 Network connectivity issues
Sep 07 2026 16:48:14 10.14.119.222 Security breaches
Sep 07 2026 17:30:32 10.14.109.102 Port Scanning Activity
Sep 07 2026 23:00:26 10.14.227.119 Data Exfiltration Attempts
Sep 07 2026 23:58:40 10.14.170.169 Security breaches
```

La lista completa esta en out/range607.txt.

Resultado guardado en out/range607.txt

Copia guardada en out/range607_log1.txt

Opcion: 2

```
Hay 6818 registros ordenados.
Los datos van de Sep 08 2024 00:22:43 a Sep 07 2026 23:58:40.
Las fechas que escribas no tienen que existir en el archivo.
Fecha de inicio: Jul 11 2025 00:00:00
Fecha de fin: Jul 11 2025 09:00:00
```

Registros encontrados: 0

```
No hay registros en ese rango. Revisa que las fechas esten dentro del periodo que cubre el archivo.
Resultado guardado en out/range607.txt
Copia guardada en out/range607_log1.txt
```

Captura P35 y P36: fechas inválidas e inicio posterior al fin

Opcion: 2

```
Hay 6818 registros ordenados.
Los datos van de Sep 08 2024 00:22:43 a Sep 07 2026 23:58:40.
Las fechas que escribas no tienen que existir en el archivo.
Fecha de inicio: Foo 32 2025 25:61:00
Fecha invalida. Usa el formato hh:mm dd aaaa hh:mm:ss. por ejemplo Oct 02 2024 23:04:24.
Fecha de inicio: Feb 20 2025 10:00:00
Fecha invalida. Usa el formato hh:mm dd aaaa hh:mm:ss. por ejemplo Oct 02 2024 23:04:24.
Fecha de inicio: Dec 16 2025 00:00:00
Fecha de fin: Jan 15 2025 00:00:00
La fecha de inicio no puede ser posterior a la de fin.
Fecha de inicio: Dec 16 2025 00:00:00
Fecha de fin: Dec 16 2025 23:59:59
```

Captura P37: wc -l de out/range607.txt

```
diegocontreras@Diegos-MacBook-Pro-101 Evidence1 % head -3 out/output607.txt
tail -3 out/output607.txt
wc -l out/output607.txt
wc -l out/range607.txt
Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication
Sep 08 2024 04:37:37 10.14.248.79 Configuration errors
Sep 08 2024 04:44:15 10.14.135.12 Application error
Sep 07 2026 17:30:32 10.14.109.102 Port Scanning Activity
Sep 07 2026 23:00:26 10.14.227.119 Data Exfiltration Attempts
Sep 07 2026 23:58:40 10.14.170.169 Security breaches
6818 out/output607.txt
13 out/range607.txt
diegocontreras@Diegos-MacBook-Pro-101 Evidence1 %
```

Captura P38: ls -R out con las cuatro copias identificadas

```
diegocontreras@Diegos-MacBook-Pro-101 Evidence1 % ls -R out
corridas.csv          output607_log1.txt    output607_log2.txt    output607.txt          range607_log1.txt    range607_log2.txt    range607.txt
diegocontreras@Diegos-MacBook-Pro-101 Evidence1 %
```

Captura P39: compilación con -Wall -Wextra sin advertencias

```
diegocontreras@Diegos-MacBook-Pro-101 Evidence1 % g++ -std=c++20 -O2 -Wall -Wextra -Iinclude src/*.cpp -o build/app && echo "compilo sin advertencias"
diegocontreras@Diegos-MacBook-Pro-101 Evidence1 %
```

Captura P40 y P41: error desde la carpeta equivocada y el script run.sh

```
diegocontreras@Diego-MacBook-Pro-101: Evidencias1 % clear
cd ~/Developer/C.HTY.TC1031.607.2613
./Evidencias/Evidencias1/build/app
=====
1. Ordenar un archivo
2. Buscar por rango de fechas
3. Ver historial de corridas
0. Salir
Opcion: 1
Que archivo quieres usar?
1. log607-1.txt (desordenado)
2. log607-2.txt (casi ordenado)
0. Regresar al menu
Archivo: 1
Error: no se pudo abrir el archivo "data/log607-1.txt".
Revisa que exista y que lo estes corriendo desde la carpeta Evidencias1.

=====
Finalizador de logs 607
=====
1. Ordenar un archivo
2. Buscar por rango de fechas
3. Ver historial de corridas
0. Salir
Opcion: 0
Hasta luego.
diegocontreras@Diego-MacBook-Pro-101 C.HTY.TC1031.607.2613 % ./Evidencias/Evidencias1/scripts/run.sh

=====
Finalizador de logs 607
=====
1. Ordenar un archivo
2. Buscar por rango de fechas
3. Ver historial de corridas
0. Salir
Opcion: 0
Hasta luego.
diegocontreras@Diego-MacBook-Pro-101 C.HTY.TC1031.607.2613 %
```

Resultados de tiempos

Metodologia. Las 16 corridas se midieron en la misma sesion del programa, en una MacBook con macOS, compilando una sola vez con `g++ -std=c++20 -O2`. Es una corrida por combinacion de algoritmo y archivo, sobre los mismos 6,818 registros. El cronometro (steady clock) rodea unicamente la llamada al algoritmo: no incluye la lectura del archivo, la copia de los datos ni la escritura de la salida. Antes de medir se cerraron los servidores de desarrollo que estaban corriendo.

Algoritmo	log607-1 (ms)	log607-2 (ms)	log607-1 entre log607-2
Bubble sort	105.697	0.290	364.47
Selection sort	21.335	22.074	0.97
Insertion sort	126.185	0.962	131.17
Merge sort	3.891	3.032	1.28
Quick sort (mediana de tres)	0.953	0.675	1.41
Swap sort	87.424	15.488	5.64
Shell sort	7.032	5.779	1.22
Quick sort (pivote al final, variante)	0.966	37.974	0.025

La ultima columna es el tiempo en log607-1 dividido entre el de log607-2. Un numero mayor que 1 significa que el algoritmo fue mas rapido con el archivo casi ordenado, y uno menor que 1 que fue mas lento con el.

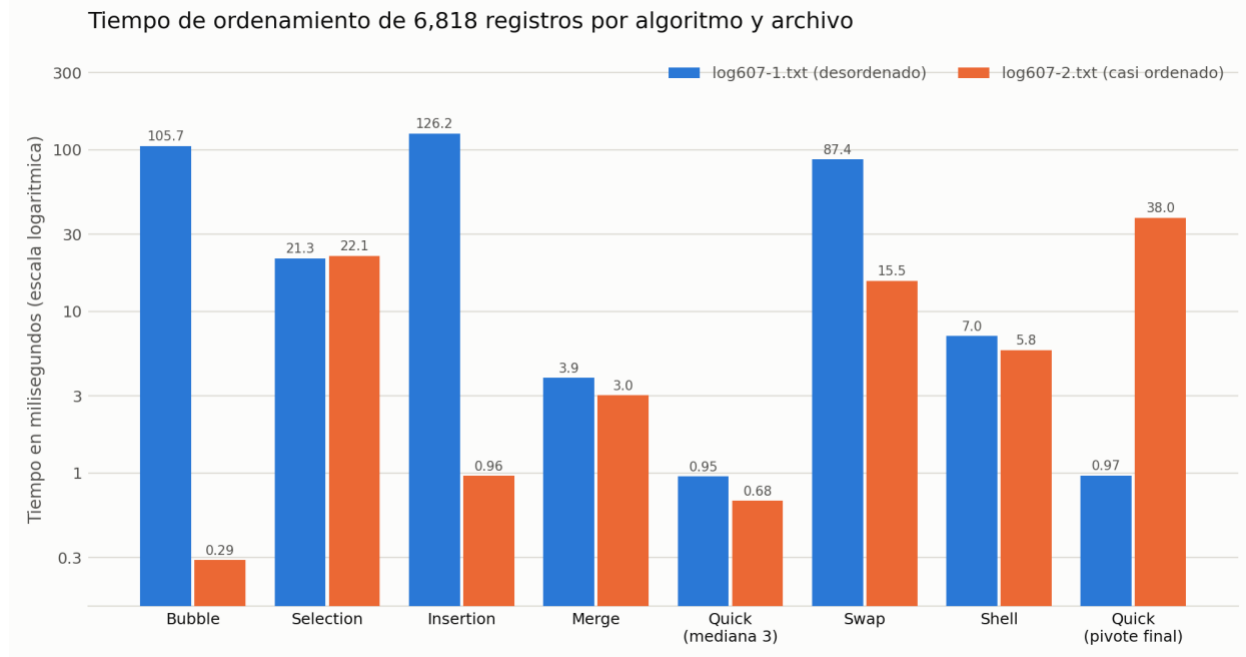


Figura 1. Tiempo de ordenamiento de los 6,818 registros por algoritmo y archivo. El eje vertical va en escala logarítmica porque los tiempos van de 0.29 ms a 126 ms y en escala normal las barras mas chicas no se distinguirían.

Conclusión

Hallazgos principales. En el archivo desordenado los más rápidos fueron los dos quick sort, con 0.953 ms la versión de mediana de tres y 0.966 ms la variante de pivote al final, seguidos de merge sort con 3.891 ms y shell sort con 7.032 ms. Los más lentos fueron insertion sort con 126.185 ms, bubble sort con 105.697 ms y swap sort con 87.424 ms. En el archivo casi ordenado el panorama se invierte en parte: el más rápido fue bubble sort con 0.290 ms, seguido de quick sort con mediana de tres con 0.675 ms e insertion sort con 0.962 ms, mientras que el más lento fue la variante de pivote al final con 37.974 ms.

Qué tanto cambió cada algoritmo entre los dos archivos. Bubble sort fue 364 veces más rápido con el archivo casi ordenado e insertion sort 131 veces más rápido. Los dos tienen mejor caso $O(n)$ y peor caso $O(n^2)$, así que su enorme mejora coincide con la teoría: la bandera de salida temprana de bubble y el hecho de que insertion solo trabaja cuando hay inversiones les permiten acercarse al mejor caso. Selection sort, en cambio, tardó prácticamente lo mismo en los dos archivos, con una razón de 0.97, y eso también coincide con la teoría porque su mejor y su peor caso son ambos $O(n^2)$: siempre recorre todo lo que le falta para encontrar el mínimo, sin forma de notar que los datos ya venían ordenados. Merge sort apenas cambió, con razón 1.28, que es lo esperado de un $O(n \log n)$ en los dos casos.

El caso más interesante es quick sort. La variante con pivote al final pasó de 0.966 ms en el archivo desordenado a 37.974 ms en el casi ordenado, o sea 39 veces más lenta justo con los datos que uno pensaría que son más fáciles. Con el archivo casi ordenado el último elemento es casi siempre el mayor del pedazo, así que la partición deja todo de un lado y nada del otro, y en vez de dividir el problema a la mitad lo reduce de uno en uno: es su peor caso $O(n^2)$. La versión de la Actividad 1.5, que escoge la mediana entre el primero, el de en medio y el último, no sufre eso y se mantuvo por debajo de 1 ms en los dos archivos. Esa comparación es la demostración práctica de para qué existe la mediana de tres.

El resultado que no cuadró con la teoría. Insertion sort tardó 126.185 ms en el archivo desordenado, más que bubble sort con 105.697 ms, cuando la teoría dice que insertion debería ganarle porque hace la mitad del trabajo. La causa no está en el algoritmo sino en lo que cuesta cada operación con este tipo de dato. Cada registro guarda la línea original completa como texto. Bubble sort mueve registros con `std::swap`, que en un texto solo intercambia dos

apuntadores y cuesta lo mismo sin importar qué tan larga sea la línea. Insertion sort los mueve con una asignación, que copia el texto entero letra por letra. Como insertion hace unos 11.6 millones de movimientos en ese archivo, ese costo extra por movimiento se come toda su ventaja en comparaciones. Para comprobarlo ordenamos exactamente las mismas fechas guardadas como números simples, sin el texto: ahí insertion tardó 6.619 ms contra 31.385 ms de bubble, que es el resultado que predice la teoría. La lección es que la notación O grande cuenta operaciones suponiendo que todas cuestan igual, y en este caso comparar es barato pero mover es caro.

Sobre las predicciones. De las 16 corridas, 12 coincidieron con la predicción y 4 no. Las cuatro fallas son el mismo error y vale la pena explicarlo: en selection sort en los dos archivos, en swap sort en el casi ordenado y en la variante de quick sort en el casi ordenado predijimos lento y salieron medio. O sea que la dirección del razonamiento fue correcta, porque son los algoritmos $O(n^2)$ y efectivamente fueron de los más tardados, pero el umbral de 60 ms quedó alto para ellos. La razón es la misma del párrafo anterior: selection y swap hacen millones de comparaciones pero muy pocos movimientos, y comparar dos números es mucho más barato que copiar una línea de texto. Contar operaciones no basta si no se sabe cuánto cuesta cada una.

Hallazgos de estabilidad. Comparando la salida de cada algoritmo contra la de bubble sort sobre el mismo archivo, insertion sort y merge sort dieron cero diferencias, mientras que selection, shell, los dos quick sort y swap sort cambiaron de lugar algunas líneas. Verificamos que todas las líneas que cambiaron de posición son de las 18 fechas que aparecen repetidas, y que las ocho salidas conservan exactamente los mismos 6,818 registros. Un algoritmo estable conserva el orden en que venían dos registros con la misma fecha y hora. Eso se nota también al comparar output607_log1.txt con output607_log2.txt: tienen los mismos registros y difieren en 20 líneas, todas de fechas repetidas, porque cada salida respeta el orden de su propio archivo de entrada.

Recomendaciones. Para un log real, que normalmente llega casi ordenado porque los eventos se escriben conforme ocurren y solo se desordena cuando se juntan varios servidores o llegan registros tarde, recomendamos merge sort. No fue el más rápido, pero su tiempo es parejo sin importar cómo vengan los datos, 3.891 ms y 3.032 ms, y sobre todo es estable. La estabilidad importa de verdad aquí: si dos eventos quedaron grabados en el mismo segundo, un algoritmo inestable puede invertirlos y hacer creer que el ataque ocurrió antes que la alerta. Para un archivo que se sabe muy desordenado, quick sort con mediana de tres fue el más rápido con 0.953 ms y sería la opción si el orden entre eventos del mismo segundo no importara. Lo que no recomendamos en ningún caso es quick sort con pivote en un extremo, porque se degrada justo con el patrón de datos más común en un log.

Una mejora concreta. Cambiar las asignaciones de insertion sort por std::move lo bajó de 126.185 ms a 25.652 ms en el archivo desordenado, unas cinco veces más rápido, sin modificar la lógica del algoritmo ni su complejidad. No se aplicó en la entrega porque el código debe ser el de la versión vista en clase, pero es la mejora más rentable que encontramos y muestra que en datos que son caros de copiar conviene mover en lugar de copiar.